开卷考试速查清单

> 使用 IA-32 指令，AT&T 格式

指令对标志位的修改

指令	CF	OF	SF	ZF
ADD / SUB	按结果更新	按结果更新	更新	更新
ADC / SBB	按结果更新	按结果更新	更新	更新
INC / DEC	**保持不变**	按结果更新	更新	更新
CMP（= SUB 不写回）	按结果更新	按结果更新	更新	更新
AND / OR / XOR	清 0	清 0	更新	更新
TEST（= AND 不写回）	清 0	清 0	更新	更新
SHL / SAL	移出的最高位	移 1 位：`新 MSB ⊕ CF`；否则未定义	更新	更新
SHR	移出的最低位	移 1 位：原 MSB；否则未定义	更新	更新
SAR	移出的最低位	移 1 位：清 0；否则未定义	更新	更新
MUL（无符号，一操作数）	高半非 0 → 1，否则 0	同 CF	未定义	未定义
IMUL（一操作数）	高半非符号扩展 → 1，否则 0	同 CF	未定义	未定义
IMUL（二 / 三操作数）	结果截断 → 1，否则 0	同 CF	未定义	未定义
DIV / IDIV	未定义	未定义	未定义	未定义

加减法CF与OF

运算	标志位	什么时候置 1	本质含义
加法	**CF**	**最高位发生了进位（结果放不下）**	无符号加法溢出
加法	**OF**	**两个操作数符号相同，但结果符号不同**	有符号加法溢出
减法	**CF**	**发生了借位（被减数不够减）**	无符号减法溢出
减法	**OF**	**两个操作数符号不同，结果符号与被减数不同**	有符号减法溢出

移位指令含义总结

指令（AT&T）	含义	数据变化	CF	OF（仅 n=1 有定义）	SF / ZF
`shl $n, %dst` `sal $n, %dst`	逻辑左移 / 算术左移（等价）	低位补 0，结果写回 `%dst`	移出的**最高位**	`新 MSB ⊕ CF`	按结果更新
`shr $n, %dst`	逻辑右移	高位补 0	移出的**最低位**	**原 MSB**	按结果更新
`sar $n, %dst`	算术右移	高位补 **符号位**	移出的**最低位**	**清 0**	按结果更新
`rol $n, %dst`	循环左移	位循环左移，不经 CF	最后移出的位	`新 MSB ⊕ CF`	**不变**
`ror $n, %dst`	循环右移	位循环右移，不经 CF	最后移出的位	`新 MSB ⊕ 次高位`	**不变**
`rcl $n, %dst`	通过 CF 循环左移	CF 参与旋转	最后移出的位	同 `rol`	**不变**
`rcr $n, %dst`	通过 CF 循环右移	CF 参与旋转	最后移出的位	同 `ror`	**不变**

条件跳转速查（比较大小）

指令	条件（自然语言）	逻辑判定（EFLAGS）	适用范围
`JE`	等于	`ZF=1`	有符号/无符号相等
`JNE`	不等于	`ZF=0`	有符号/无符号不等
`JA`	无符号大于（>）	`CF=0 && ZF=0`	无符号
`JAE`	无符号大于等于（>=）	`CF=0`	无符号
`JB`	无符号小于（<）	`CF=1`	无符号
`JBE`	无符号小于等于（<=）	`CF=1 || ZF=1`	无符号
`JG`	有符号大于（>）	`ZF=0 && SF=OF`	有符号
`JGE`	有符号大于等于（>=）	`SF=OF`	有符号
`JL`	有符号小于（<）	`SF != OF`	有符号
`JLE`	有符号小于等于（<=）	`ZF=1 || SF != OF`	有符号

传参寄存器选择

整数

参数序号	使用寄存器
第 1 个	`%rdi`
第 2 个	`%rsi`
第 3 个	`%rdx`
第 4 个	`%rcx`
第 5 个	`%r8`
第 6 个	`%r9`
第 7+ 个	**栈上传递**

浮点数

参数序号	使用寄存器
第 1 个	`%xmm0`
第 2 个	`%xmm1`
第 3 个	`%xmm2`
第 4 个	`%xmm3`
第 5 个	`%xmm4`
第 6 个	`%xmm5`
第 7 个	`%xmm6`
第 8 个	`%xmm7`
第 9+ 个	栈

乘除法

乘法

指令（AT&T）	有符号	操作数形式	隐含被操作数	结果（商/积）	高半 / 余数	CF / OF	说明
`mulb r/m8`	否	1	`%al`	`%ax`	—	高 8 位 ≠0 →1	无符号 8 位乘

指令（AT&T）	有符号	操作数形式	隐含被操作数	结果（商/积）	高半 / 余数	CF / OF	说明
`mulw r/m16`	否	1	`%ax`	`%ax`	`%dx`	高 16 位≠0 →1	无符号 16 位乘
`mull r/m32`	否	1	`%eax`	`%eax`	`%edx`	高 32 位≠0 →1	无符号 32 位乘
`imulb r/m8`	是	1	`%al`	`%ax`	—	非符号扩展 →1	有符号乘
`imulw r/m16`	是	1	`%ax`	`%ax`	`%dx`	非符号扩展 →1	有符号乘
`imull r/m32`	是	1	`%eax`	`%eax`	`%edx`	非符号扩展 →1	有符号乘
`imulw r/m16, %r16`	是	2	—	`%r16`	—	截断 →1	结果截断
`imull r/m32, %r32`	是	2	—	`%r32`	—	截断 →1	结果截断
`imulw $imm, r/m16, %r16`	是	3	—	`%r16`	—	截断 →1	立即数乘
`imull $imm, r/m32, %r32`	是	3	—	`%r32`	—	截断 →1	立即数乘
`divb r/m8`	否	1	`%ax`	`%al`	`%ah`	未定义	无符号除
`divw r/m16`	否	1	`%dx:%ax`	`%ax`	`%dx`	未定义	无符号除
`divl r/m32`	否	1	`%edx:%eax`	`%eax`	`%edx`	未定义	无符号除
`idivb r/m8`	是	1	`%ax`	`%al`	`%ah`	未定义	有符号除
`idivw r/m16`	是	1	`%dx:%ax`	`%ax`	`%dx`	未定义	有符号除
`idivl r/m32`	是	1	`%edx:%eax`	`%eax`	`%edx`	未定义	有符号除

调用者与被调用者保存寄存器

寄存器	保存规则	说明
EAX	调用者保存	返回值寄存器
ECX	调用者保存	计数 / 临时
EDX	调用者保存	乘除 / 临时
EBX	**被调用者保存**	通用
ESI	**被调用者保存**	通用
EDI	**被调用者保存**	通用
EBP	**被调用者保存**	帧指针
ESP	**被调用者保存**	栈指针（必须保持）
EIP	特殊	由 `call/ret` 隐式维护
EFLAGS	调用者保存	调用后不可假定保持

栈帧处理指令

指令	栈操作	ESP 变化	EIP 变化	等价操作（语义）	典型用途
`call target`	`push 返回地址`	`esp -= 4`	`eip = target`	`push eip; jmp target`	调用函数
`leave`	恢复旧栈帧	`esp = ebp; esp += 4`	不变	`mov %ebp, %esp; pop %ebp`	函数收尾
`ret`	`pop 返回地址`	`esp += 4`	`eip = [esp]`	`pop eip`	返回调用者

对齐

数据的对齐

类型	大小（32 位）	对齐（32 位）	大小（64 位）	对齐（64 位）	说明
`char`	1	1	1	1	字节
`short`	2	2	2	2	半字
`int`	4	4	4	4	字
`long`	4	4	**8**	**8**	64 位下变大

类型	大小（32位）	对齐（32位）	大小（64位）	对齐（64位）	说明
`long long`	8	**4**	8	**8**	关键差异
指针	4	4	8	8	地址宽度
`float`	4	4	4	4	IEEE754
`double`	8	**4**	8	**8**	常考
`long double`	12 / 16	4	16	16	实现相关

结构体成员对齐

规则	说明
成员起始偏移	必须是 **该成员对齐值的整数倍**
成员间填充	编译器自动插入 padding
成员顺序	**严格按声明顺序**，不会重排
数组成员	对齐规则 = 元素类型

结构体整体对齐

项目	规则
结构体对齐值	= **所有成员中最大的对齐要求**
结构体大小	必须是 **结构体对齐值的整数倍**
尾部填充	若不足，编译器补 padding

重定位

重定位类型	位宽	写入二进制的值	是否 PC 相对	典型指令
R_386_32	32	**要访问的目标符号的最终地址**	否	`mov imm32, reg` `mov reg, [imm32]`

R_386_PC32	32	**要跳转到的目标地址 − 下一条指令的地址**	是	`call rel32`
				`jmp rel32`
				`jcc rel32`

浮点数指令

类别	指令关键字	特点
装入浮点	`FLD`	push
存储不弹	`FST`	不 pop
存储并弹	`FSTP`	pop
装入整数	`FILD`	int → float, push
存为整数	`FIST / FISTP`	float → int
复制	`FLD ST(i)`	push
交换	`FXCH`	交换
常量	`FLD*`	push